Entrega B — Topologia em contêineres (Horas 2 e 3)

Arquivo: `docker-compose.yml`. Base: Apêndice A.1, com os ajustes abaixo.

Decisões e justificativas

Decisão	Onde	Por quê
Só o Nginx publica porta (`8080:80`); Prometheus em `127.0.0.1:9090`	`ports:`	Passo 2 da Hora 2; nada administrativo exposto à rede
Versões fixadas: `nginx:1.27.5-alpine`, `prom/prometheus:v2.53.0`, `rust:1.90-slim-bookworm`, `debian:bookworm-slim`, `python:3.12-alpine`	`image:` / `Dockerfile`	Regra do enunciado (nada de `latest`); reprodutível
Imagens próprias com tag `0.1.0`	`image: sentinel-*:0.1.0`	Saber qual versão está rodando
`x-rust-app` (âncora YAML) para A e B	topo do arquivo	A e B idênticos, só muda `NODE_NAME` — evita divergência acidental
Healthcheck do Docker = **liveness** (`/health/live`) nos nós; `/nginx/live` local no Nginx	`healthcheck:`	Liveness não pode depender de dependências (achado 6)
`restart: on-failure:3`	todos	Reinicia só se o processo **morrer**, no máximo 3 vezes: evita reinício em loop. `docker compose stop` manual não é desfeito
Processos sem root (`USER app` / `USER wd`)	Dockerfiles	Boa prática; nada exige root
Compose `name: sentinel`	topo	Nomes previsíveis: `sentinel-rust-a-1`, rede `sentinel_sentinel`
Watchdog com `SINTETICO=http://nginx:80/`	`environment`	Teste sintético separado de readiness (regra de ouro 2)
Build em 2 estágios no Rust, com cache de dependências	`rust-app/Dockerfile`	Imagem final de 34,5 MB; rebuild rápido

Kit implementado (Apêndice B)

O kit do professor não estava no repositório; implementamos as rotas do Apêndice B:

Rota	Comportamento	Observação
`GET /`	`atendido por <nó> request_id=<id>`; aplica atraso; **503 se não pronto**	503 faz o Nginx refazer em outro nó
`GET /erro`	500	Nginx **não** refaz 500

Rota	Comportamento	Observação
`GET /health/live`	200 sempre que o processo responde; **nunca** atrasa	
`GET /health/ready`	200/503 conforme prontidão; aplica atraso	Nó lento parece lento para o watchdog
`GET /metrics`	texto Prometheus	bloqueado no Nginx
`GET /admin/ready?v=0|1`	liga/desliga prontidão	bloqueado no Nginx; usar `scripts/admin.sh`
`GET /admin/atraso?ms=N`	atraso artificial (teto 10 000 ms)	idem

Extras opcionais do Apêndice B já feitos: **request_id nos logs do Rust** e **teste sintético** do caminho completo (watchdog + `scripts/teste_sintetico.sh`).

Evidências

- Estado: `evidencias/preliminar/01_compose_ps.txt` (5 serviços `Up (healthy)`, portas publicadas, IPs).
- Primeiras linhas de log: `02_logs_iniciais.txt`.
- Health checks 200 e 503: `03_health_checks.txt`.

Limitações

- Se não der para o professor compilar Rust na máquina da turma: o build leva ~1–2 min e precisa de acesso ao crates.io. Plano B: levar as imagens já construídas (`docker save sentinel-rust-app:0.1.0 sentinel-watchdog:0.1.0 -o imagens.tar`).
- Uma só máquina, um só Docker Engine (ver mapa de SPOF).